Project Evaluation - Milestone Reports

Milestone 1 - Sequential CPU Implementation

The evaluation began with testing the CPU-based implementation of sparse matrix-sparse matrix multiplication. Using the -a flag, the program executed the sequential path through spmspm_cpu0. The dataset used for testing, dataset1, included two sparse matrices in COO format and a reference output matrix. The program ran successfully, loading all files without issue and producing output without any crashes or runtime errors.

The CPU implementation completed execution in approximately 234.13 milliseconds on Octopus, AUB's HPC cluster. While the number of non-zero elements in the output did not match the reference output, the test still provided a reliable baseline to evaluate GPU performance later on. The main objective here was to validate functionality and establish a consistent environment for timing future runs, both of which were achieved.

Milestone 2 - Basic GPU Implementation (Kernel 0)

The second evaluation focused on the baseline GPU implementation using kernel0.cu. This version introduced parallelism using CUDA but did not yet incorporate specific performance optimizations. The test was run with the -0 flag and followed the same setup as the CPU version. All inputs were correctly loaded, and the GPU memory allocation and data transfer processes completed without any problems.

The log confirmed that the program successfully ran kernel 0 on a GPU-enabled node. Although no execution time was printed, the run verified that the CUDA setup was functioning and that the basic parallel version was working correctly. This step marked the transition from sequential to parallel

execution and provided a solid foundation for evaluating subsequent optimizations.

Milestone 3 - First GPU Optimization (Kernel 1)

In this phase, the project introduced its first optimized GPU implementation. The kernel, kernel1.cu, improved upon the initial GPU version by reorganizing how threads access and process data. The test was executed with the -1 flag and used the same dataset and structure as before. Input files were loaded correctly, memory allocation was successful, and the program ran to completion without errors.

This version emphasized more efficient use of GPU resources by optimizing memory access and thread distribution. Although the kernel's runtime was not captured, its stability and successful integration into the program flow were clear indicators of its functionality. Kernel 1 represents a step forward in refining performance and minimizing inefficiencies identified in the baseline version.

Milestone 4 - Second GPU Optimization (Kernel 2)

The fourth milestone evaluated kernel2.cu, a further optimized version that built on the enhancements introduced in kernel 1. This kernel aimed to improve memory coalescing, reduce thread divergence, and increase throughput by tightening how threads and blocks were organized. Execution followed the same flow as in previous tests, using the -2 flag to trigger the kernel.

Everything ran as expected. GPU memory was allocated successfully, the matrices were transferred without issue, and the kernel executed cleanly. No crashes or memory errors occurred. With kernel 2 in place, the system demonstrated the ability to support more aggressive optimizations while maintaining stability and correctness throughout the execution flow.

Milestone 5 - Third GPU Optimization (Kernel 3)

The final milestone tested kernel3.cu, the most optimized kernel in the project. This version introduced advanced CUDA techniques aimed at maximizing GPU efficiency, such as reducing synchronization overhead, optimizing memory access patterns, and improving occupancy. The program was executed using the -3 flag, and the same dataset used in earlier milestones.

As with previous runs, the input matrices were loaded successfully, GPU memory was allocated, and the kernel completed execution without any errors. The program's behavior confirmed that kernel 3 had been fully integrated and was running as intended. This final test concluded the optimization pipeline, showcasing a complete and functional GPU-accelerated implementation of sparse matrix multiplication.